Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Práctica 01

Conversión de Minúsculas a Mayúsculas en Ensamblador x86

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: April 24, 2026

Contents

1 Introducción **2**

2 Organización del programa **2**

3 Código fuente **4**

3.1 Código completo (conversion.asm) 4

4 Instrucciones x86 utilizadas **5**

5 El rol de los registros **5**

6 Ejemplo de ejecución **6**

7 Repositorio GitHub **6**

8 Conclusiones **6**

1 Introducción

Esta práctica consiste en escribir un programa en **lenguaje ensamblador x86** que tome una cadena de texto y convierta todas sus letras minúsculas en mayúsculas, sin usar ninguna función externa: todo el trabajo lo hace el programa por sí solo, directamente sobre la memoria de la computadora.

El objetivo principal no es simplemente "poner letras en mayúscula", sino entender cómo una computadora representa y manipula texto a su nivel más básico. Para lograrlo, es necesario comprender dos conceptos fundamentales: la **tabla ASCII** y el **acceso directo a memoria**.

¿Qué es la tabla ASCII?

Las computadoras no almacenan letras directamente; almacenan números. La tabla ASCII (por sus siglas en inglés: *American Standard Code for Information Interchange*) es un acuerdo universal que asigna un número a cada carácter. Por ejemplo:

Carácter	Valor decimal	Tipo
`A`	65	Mayúscula
...	...	...
`Z`	90	Mayúscula
`a`	97	Minúscula
...	...	...
`z`	122	Minúscula

Table 1: Fragmento relevante de la tabla ASCII

La observación clave es que **toda letra minúscula tiene un valor exactamente 32 unidades mayor que su correspondiente mayúscula**. Por lo tanto, para convertir de minúscula a mayúscula basta con **restar 32** al número que representa al carácter:

$$\text{mayúscula} = \text{minúscula} - 32 \tag{1}$$

2 Organización del programa

El proyecto consta de un único archivo:

Archivo	Qué hace
`conversion.asm`	Declara la cadena de texto en memoria, la recorre carácter por carácter y convierte las minúsculas a mayúsculas.

Table 2: Archivo del proyecto

El programa se divide internamente en dos secciones:

- **Sección `.data`:** es donde se declara y almacena la cadena de texto inicial en la memoria del programa.
- **Sección `.code`:** contiene las instrucciones que el procesador ejecuta una por una para recorrer y modificar esa cadena.

El flujo general del programa es el siguiente:

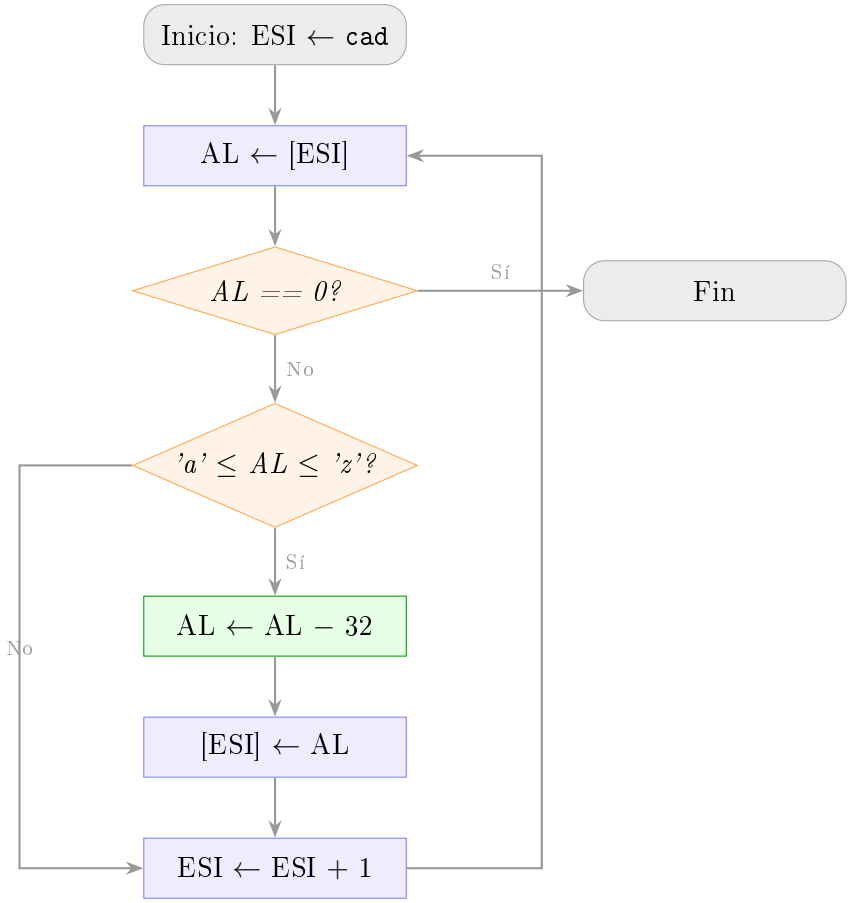

Figure 1: Diagrama de flujo del algoritmo de conversión

1. El programa apunta al primer carácter de la cadena.
2. Lee el carácter y verifica si es una letra minúscula (valor entre 97 y 122).
3. Si lo es, le resta 32 y escribe el resultado de vuelta en la misma posición de memoria.
4. Avanza al siguiente carácter y repite desde el paso 2.
5. Cuando encuentra el carácter `0` (fin de cadena), termina la ejecución.

3 Código fuente

3.1 Código completo (conversion.asm)

A continuación se presenta el programa completo:

```
.586
.model flat, stdcall

; *** SECCION DE DATOS ***
; Aqui se reserva espacio en memoria para la cadena.
; El 0 al final es el marcador de fin de cadena.
.data
    cad BYTE "Hola Mundo", 0

; *** SECCION DE CODIGO ***
.code

EXTERN ExitProcess@4 : PROC

main PROC

    mov esi, offset cad      ; ESI apunta al primer caracter

; --- CICLO PRINCIPAL ---
comparar_caracter:

    mov al, [esi]            ; AL = caracter actual
    cmp al, 0                ; ?Es el fin de la cadena?
    je  finalizar            ; Si es 0, terminar

    ; Verificar si el caracter es una minuscula
    cmp al, 'a'              ; ?Es menor que 'a' (97)?
    jb  siguiente            ; Si es menor, no es minuscula
    cmp al, 'z'              ; ?Es mayor que 'z' (122)?
    ja  siguiente            ; Si es mayor, no es minuscula

    ; CONVERSION: restar 32 convierte minuscula a mayuscula
    sub al, 32
    mov [esi], al            ; Guardar la mayuscula en memoria

siguiente:
    inc esi                  ; Avanzar al siguiente caracter
    jmp comparar_caracter    ; Repetir el ciclo

finalizar:
    push 0
    call ExitProcess@4       ; Cerrar el programa

main ENDP
END main
```

Listing 1: conversion.asm — Conversión de minúsculas a mayúsculas

4 Instrucciones x86 utilizadas

La siguiente tabla resume todas las instrucciones ensamblador empleadas en el programa, con una explicación accesible de su función:

Instrucción	**Qué hace en términos simples**
`MOV`	Copia un valor de un lugar a otro (de memoria a registro o viceversa).
`CMP`	Compara dos valores restándolos internamente, sin guardar el resultado; solo activa señales para las instrucciones de salto.
`JE`	Salta a otra parte del código si los dos valores comparados con `CMP` fueron iguales.
`JB`	Salta si el primer valor comparado fue menor que el segundo (sin signo).
`JA`	Salta si el primer valor comparado fue mayor que el segundo (sin signo).
`JMP`	Salta incondicionalmente a otra parte del código (siempre).
`SUB`	Resta el segundo operando al primero y guarda el resultado en el primero.
`INC`	Incrementa en 1 el valor del operando; equivale a `+1`.
`PUSH`	Coloca un valor en la pila de memoria.
`CALL`	Llama a una función o procedimiento externo.

Table 3: Instrucciones x86 utilizadas en el programa

5 El rol de los registros

Un **registro** es una pequeña celda de memoria ubicada dentro del propio procesador, de acceso inmediato (mucho más rápido que la RAM). En este programa se usan dos registros principales:

- **ESI** (*Extended Source Index*): actúa como un **puntero** o "cursor" que va avanzando por la cadena. Guarda la dirección de memoria del carácter que se está procesando en este momento.

- **AL**: es la mitad baja del registro `EAX`, de 8 bits. Es perfecto para trabajar con caracteres ASCII, ya que un byte (8 bits) puede representar valores del 0 al 255, cubriendo toda la tabla ASCII.

La modificación ocurre **directamente en memoria**: el programa lee el carácter a `AL`, lo modifica ahí, y escribe el resultado de vuelta a la misma posición. No se crea una copia de la cadena; se trabaja sobre la original.

6 Ejemplo de ejecución

Al ensamblar y ejecutar el programa en un depurador, se puede verificar que la región de memoria donde estaba almacenada la cadena `"Hola Mundo"` es sobreescrita con `"HOLA MUNDO"`:

```
Antes de ejecutar:  48 6F 6C 61 20 4D 75 6E 64 6F 00
                    H  o  l  a     M  u  n  d  o  \0
```

Figure 2: Registros de memoria con "Hola Mundo"

```
Después de ejecutar: 48 4F 4C 41 20 4D 55 4E 44 4F 00
                     H  O  L  A     M  U  N  D  O  \0
```

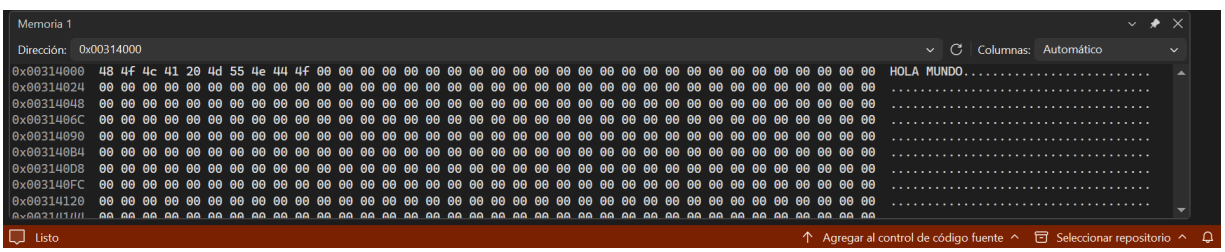

Figure 3: Registros de memoria con "HOLA MUNDO"

7 Repositorio GitHub

https://github.com/7mo-ArquitecturaComputadoras/Practica01_ConversionMinusculasMayusculas

8 Conclusiones

- El programa demuestra que **trabajar con texto en ensamblador es trabajar con números**: las letras son valores enteros en la tabla ASCII, y su manipulación se reduce a operaciones aritméticas simples.
- La diferencia de 32 entre las mayúsculas y las minúsculas en la tabla ASCII no es accidental; fue un diseño deliberado que facilita exactamente este tipo de conversión con una sola resta.

- El uso de **registros como punteros** (`ESI`) es un patrón fundamental en ensamblador para recorrer estructuras de datos en memoria, equivalente a un índice de arreglo en lenguajes de alto nivel.
- El **filtro de rango** (verificar que el valor esté entre 97 y 122) es indispensable: sin él, el programa restaría 32 a espacios, números y mayúsculas, corrompiéndolos.
- Esta práctica evidencia lo que sucede "detrás de escenas" cuando un lenguaje como Python o Java convierte texto a mayúsculas con una función: al fondo, el procesador realiza exactamente esta misma secuencia de comparaciones y restas.